

## SOFTWARE ENGINEERING PROJECT

# AI-powered Platform

## Connecting Film Photography Enthusiasts with Film Processing Labs

Proposal, System Design, and Prototype Handover

| Team member           | Primary work package              |
|-----------------------|-----------------------------------|
| Nguyễn Thái Gia Bảo   | Context and business requirements |
| Lê Thị Như Quân       | Architecture and database         |
| Huỳnh Gia Hợp         | Backend and API                   |
| Nguyễn Đào Quốc Khánh | Mobile and web applications       |
| Nguyễn Thành Toàn     | AI, testing, and deployment       |

Submission snapshot: 17 September 2026

<https://github.com/thanhtoan101/CNPM>

# Report Scope

This report consolidates the team’s proposal, requirements, architecture, and mobile/web design. Statements using “shall”, “will”, and “proposed” define the intended platform and its acceptance criteria; they do not imply that every feature has already been implemented or deployed.

The repository contains a React/TypeScript web prototype, a Flutter mobile prototype, an Express backend, and a separate Python AI prototype. The frontend demonstration uses local data. A working user interface does not establish successful payment, real cloud scan delivery, cross-device synchronization, or production authentication. The final chapter records the implementation boundaries and a reproducible demonstration checklist.

Task ownership describes agreed responsibilities. Git history and Jira work items provide change evidence; a planned allocation of 20% per member is not a measured contribution percentage. Evaluation must consider the content, tests, and documented work as well as commit counts.

# Contents

|                                                             |           |
|-------------------------------------------------------------|-----------|
| <b>Report Scope</b>                                         | <b>i</b>  |
| <b>1 Project Proposal and Requirements</b>                  | <b>1</b>  |
| 1.1 Context . . . . .                                       | 1         |
| 1.2 Proposed Solutions . . . . .                            | 2         |
| 1.3 Functional Requirements . . . . .                       | 4         |
| 1.4 Use Case Specification and Business Rules . . . . .     | 7         |
| 1.5 Technical Requirements . . . . .                        | 11        |
| 1.6 Theory and Practical . . . . .                          | 14        |
| 1.7 Expected Deliverables . . . . .                         | 17        |
| 1.8 Proposed Tasks and Responsibilities . . . . .           | 19        |
| <b>2 System and Interface Design</b>                        | <b>23</b> |
| 2.1 System Architecture . . . . .                           | 23        |
| 2.2 Technology Stack . . . . .                              | 24        |
| 2.3 Implemented Data Model and API Contracts . . . . .      | 29        |
| 2.4 Mobile and Web Design . . . . .                         | 32        |
| <b>3 Prototype Handover and Verification</b>                | <b>36</b> |
| 3.1 Repository Deliverables . . . . .                       | 36        |
| 3.2 Demonstration Procedure . . . . .                       | 36        |
| 3.3 Acceptance and Remaining Integration Work . . . . .     | 37        |
| 3.4 Contribution and Submission Evidence . . . . .          | 37        |
| 3.5 Verification Snapshot . . . . .                         | 38        |
| 3.6 Requirement-to-Code Examples . . . . .                  | 38        |
| 3.7 Workflow and Sequence Diagrams . . . . .                | 39        |
| 3.8 Known Limits and Demonstration Talking Points . . . . . | 40        |
| 3.9 Measurable Quality Acceptance Plan . . . . .            | 41        |
| 3.10 Acceptance Traceability . . . . .                      | 41        |
| 3.11 Release Checklist and Risks . . . . .                  | 42        |
| 3.12 Glossary and Evidence Sources . . . . .                | 43        |

# List of Figures

|     |                                                                                                            |    |
|-----|------------------------------------------------------------------------------------------------------------|----|
| 1.1 | Business use case associations; supporting AI and storage services are shown in the architecture . . . . . | 7  |
| 2.1 | Proposed system architecture: shared API, data, storage, search, and AI services . . . . .                 | 23 |
| 2.2 | Database and Digital Storage Architecture . . . . .                                                        | 28 |
| 2.3 | ERD derived from the current five-table bootstrap schema . . . . .                                         | 29 |
| 3.1 | Implemented local web workflow; no cloud upload or customer message is sent . . . . .                      | 39 |
| 3.2 | Implemented state persistence, validation and error handling in the web demo . . . . .                     | 39 |

# List of Tables

|     |                                                        |    |
|-----|--------------------------------------------------------|----|
| 2.1 | Proposed Technology Stack . . . . .                    | 25 |
| 2.4 | Main Photographer Mobile Application Screens . . . . . | 33 |
| 2.5 | Frontend Architecture Layers . . . . .                 | 34 |
| 3.1 | Evidence and implementation boundaries . . . . .       | 36 |

# Chapter 1

## Project Proposal and Requirements

### 1.1 Context

#### 1.1.1 Background

In recent years, film photography has experienced a significant revival worldwide, including in Vietnam. Increasing numbers of photography enthusiasts, artists, and collectors are returning to analog photography because of its distinctive aesthetic qualities, creative experience, and tangible value.

Unlike digital photography, film photography requires a more deliberate shooting process and produces unique visual characteristics that cannot always be replicated by digital filters.

As the analog photography community continues to grow, the demand for supporting services such as film developing, film scanning, photo printing, film preservation, and digital archiving has also increased. However, the service ecosystem surrounding film photography remains highly fragmented. Photographers often need to search for Film Labs through social media platforms, online communities, messaging applications, or personal recommendations.

Information regarding service availability, pricing, turnaround time, supported film formats, processing quality, and customer reviews is frequently scattered across multiple channels.

#### 1.1.2 Current Problems

Despite the growing interest in film photography, several challenges remain:

- Difficulty in discovering and comparing Film Labs.
- Lack of transparent information about pricing and services.
- Inconvenient service booking and order management.
- Limited real-time tracking of film processing orders.
- Fragmented communication between photographers and Film Labs.
- Difficulty in preserving and organizing digital film archives.
- Lack of a centralized community platform for sharing knowledge and experiences.

### 1.1.3 Project Motivation

The project is motivated by the need to create a centralized digital platform that connects film photography enthusiasts with Film Processing Labs.

By integrating service discovery, booking, order tracking, digital archiving, community interaction, and AI-powered assistance into a single platform, the system aims to simplify the overall film photography workflow.

The proposed platform will reduce the difficulty of finding reliable Film Labs while helping service providers manage their operations more efficiently. At the same time, photographers can access information, compare services, track orders, store digital scans, and interact with the film photography community through one unified system.

### 1.1.4 Project Objective and Significance

The main objective of this project is to develop an AI-powered platform that connects film photography enthusiasts with Film Processing Labs.

The platform aims to:

- Connect photographers with suitable Film Labs.
- Provide transparent information about services and pricing.
- Support service booking and order management.
- Enable real-time order tracking.
- Provide digital film archive management.
- Support community interaction and knowledge sharing.
- Integrate AI-powered features to improve user experience.

The project is expected to contribute to the digital transformation of the film photography service ecosystem while preserving and supporting the growth of the analog photography community.

## 1.2 Proposed Solutions

### 1.2.1 Overall Solution

The proposed solution is an AI-powered digital platform that connects film photography enthusiasts with Film Processing Labs through a centralized ecosystem.

The platform provides a unified environment where photographers can discover Film Labs, compare services, make bookings, track processing orders, manage digital film archives, and interact with the film photography community.

For Film Lab owners, the system provides management tools for handling services, pricing, orders, customers, and operational workflows.

### 1.2.2 System Roles

The platform supports the following main user roles:

- **Photographer:** Users who search for Film Labs, book services, track orders, manage digital archives, and interact with the community.
- **Film Lab Owner:** Service providers who manage Film Lab profiles, services, pricing, customer orders, and processing workflows.
- **Film Lab Employee:** Staff who receive film, update assigned processing stages and review scans under the owner's permissions.
- **Photography Expert / Community Member:** Contributors who publish knowledge, join discussions and organize workshops or photowalks.
- **Delivery Partner:** Users or service providers responsible for pickup and delivery requests.
- **System Administrator:** Users responsible for managing the overall platform, users, services, and system operations.

The AI Service, cloud storage and logistics integration are supporting system actors, not human login roles. The current backend role vocabulary is narrower than this target actor catalogue; the implementation contract chapter records that distinction.

### 1.2.3 System Features

The proposed platform integrates several key features:

- Film Lab discovery and comparison.
- Service booking and order management.
- Pickup and delivery request management.
- Real-time order tracking.
- Digital film archive management.
- Marketplace and community interaction.
- AI-powered photography assistance and recommendations.

### 1.2.4 Expected Benefits

The proposed solution is expected to provide benefits for both photographers and Film Labs.

For photographers, the platform simplifies the process of finding reliable Film Labs, comparing services, making bookings, and tracking film processing orders.

For Film Labs, the platform provides a centralized management system that improves service visibility, customer communication, and operational efficiency.

For the film photography community, the platform creates a digital ecosystem that supports knowledge sharing, community interaction, and the long-term preservation of film photography.

## 1.3 Functional Requirements

### 1.3.1 Business Core Flows

The functional requirements of the proposed platform are described through the main business flows of the system. These flows show how photographers, Film Labs, community members, and AI-supported services interact with each other on the platform.

#### Core Flow 1: Intelligent Film Lab Discovery

Photographers can search for Film Labs based on location, supported film formats, available services, pricing, turnaround time, community ratings, or special processing techniques. The platform collects information from multiple labs and uses AI recommendation methods to suggest the most suitable options based on user preferences and previous activities.

#### Core Flow 2: Service Booking and Film Collection

After selecting a Film Lab, photographers create service orders by providing film type, processing options, scanning quality, printing requirements, and other requests. Customers can either bring film rolls directly or request a pickup service through delivery partners. The Film Lab then checks the order and arranges processing.

#### Core Flow 3: Film Processing Lifecycle Management

Film Lab operators manage all stages of processing, including receiving film rolls, chemical development, washing, drying, scanning, image preparation, and final quality checking. Each step is recorded and synchronized with the customer application so that users can follow the order progress in real time.

#### Core Flow 4: Digital Delivery and Film Archive Management

After processing is completed, Film Labs upload digital scan files to secure cloud storage. Customers receive notifications and can view, download, organize, and store their scanned photographs in a personal Digital Film Archive. The system also keeps metadata such as film stock, camera model, lens, shooting date, processing method, and scanning specifications for later reference.

#### Core Flow 5: Marketplace and Equipment Exchange

Community members can post listings to buy, sell, or exchange analog cameras, film rolls, lenses, and photography accessories. The platform supports listing management, searching, seller ratings, transaction tracking, and communication between buyers and sellers to create a clearer and more trustworthy trading environment.

#### Core Flow 6: Knowledge Sharing and Community Engagement

Photography experts and experienced community members can share tutorials, technical articles, equipment reviews, lab recommendations, and photography experiences. Users can also join discussions, register for workshops or photowalk events, and contribute to the common knowledge base of the platform.

### 1.3.2 AI-powered Personalized Assistance

Artificial Intelligence analyzes service history, user preferences, community feedback, and knowledge resources to provide personalized Film Lab recommendations, evaluate scan quality, answer technical questions, and recommend suitable learning resources through an intelligent assistant.

The personalized assistance process includes the following main steps:

1. The photographer enters a question or selects a support topic.
2. The system collects relevant information such as user preferences, previous service orders, saved Film Labs, and community feedback.
3. The AI analyzes the request and returns suitable recommendations with short explanations.
4. The photographer reviews the response, asks a follow-up question, or selects a recommended action.
5. The system records user feedback to improve future recommendations.

If there is not enough user history, the AI asks for basic preferences and provides general suggestions. If the AI service is unavailable, users can still use the normal search, filtering, and knowledge-base functions.

### Main Functional Modules

The platform is divided into the following main functional modules. Each module supports one or more business core flows described above.

#### Module 1. Account and Profile Management

- Supports account registration, sign-in, sign-out, and account recovery.
- Manages personal information, photography interests, service preferences, and delivery addresses.
- Supports the Photographer, Film Lab Operator, Photography Expert, and System Administrator roles.

#### Module 2. Film Lab Discovery and Recommendation

- Manages Film Lab profiles, services, prices, supported film formats, turnaround times, and locations.
- Allows photographers to search, filter, compare, rate, and review Film Labs.
- Recommends suitable Film Labs based on preferences, previous activity, and community feedback.

#### Module 3. Service Booking and Film Collection

- Allows photographers to select processing, scanning, printing, and other service options.
- Calculates estimated costs and supports direct delivery or pickup by a Delivery Partner.
- Creates service orders with order codes, confirmation information, and film handling instructions.

**Module 4. Film Processing Lifecycle Management**

- Allows Film Lab operators to receive, verify, and schedule film orders.
- Records development, washing, drying, scanning, image preparation, and quality-check stages.
- Synchronizes timestamps, notes, and order status so photographers can track progress in real time.

**Module 5. Digital Delivery and Film Archive**

- Uploads completed scans to secure cloud storage and links them to the correct order.
- Allows photographers to preview, download, organize, tag, and favorite photographs.
- Stores film, camera, lens, processing, and scanning metadata with access control.

**Module 6. Marketplace and Equipment Exchange**

- Allows users to create and manage listings for analog photography equipment and film.
- Supports search, buyer-seller communication, purchase or exchange requests, and transaction tracking.
- Supports ratings, cancellation, dispute reporting, and administrator review.

**Module 7. Knowledge Sharing and Community**

- Allows users to publish tutorials, articles, reviews, and photography experiences.
- Supports discussions, comments, questions, workshops, photowalks, and event registration.
- Supports content reporting and moderation to maintain community quality.

**Module 8. AI-powered Personalized Assistance**

- Recommends Film Labs and learning resources based on user needs.
- Identifies common scan-quality problems such as dust, color cast, low contrast, or incorrect exposure.
- Answers technical questions using the platform knowledge base and collects feedback for later recommendations.

**Module 9. Notification and Administration**

- Notifies users about orders, pickup, processing progress, digital delivery, marketplace transactions, and events.
- Allows administrators to manage users, Film Labs, services, reported content, and disputes.
- Records important system activities and provides basic management reports.

## 1.4 Use Case Specification and Business Rules

The six use cases refine the business flows into reviewable contracts. They describe the target platform. A success condition in this section is an acceptance requirement, not evidence that the integration already exists. The identifiers UC-01–UC-06 are local report identifiers and do not create additional Jira tasks.

### 1.4.1 Use Case Overview

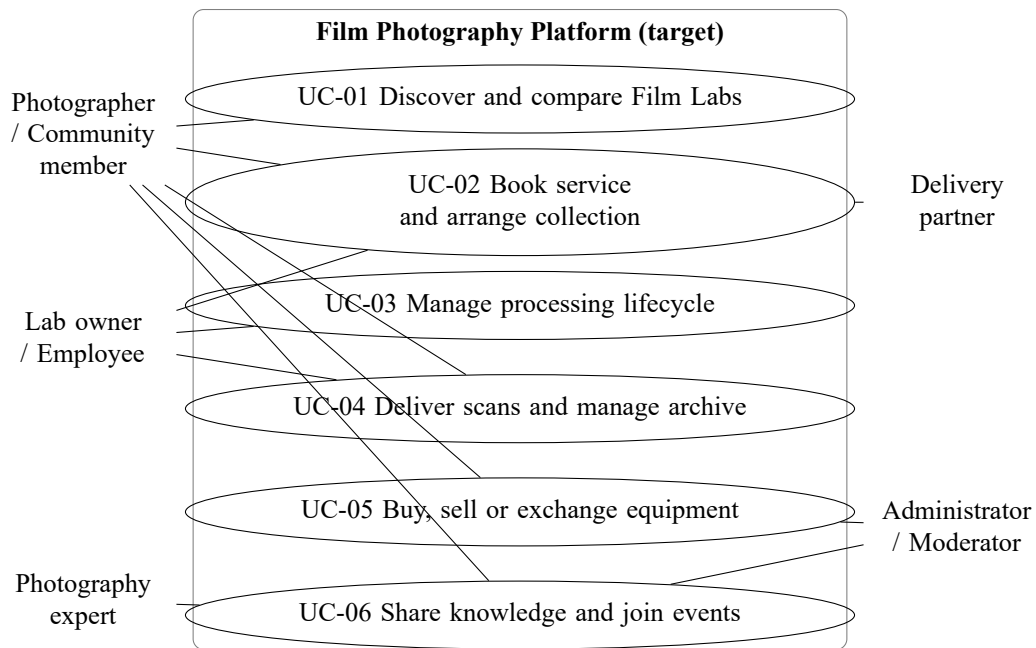


Figure 1.1: Business use case associations; supporting AI and storage services are shown in the architecture

### 1.4.2 UC-01: Discover and Compare Film Labs

**Primary actor:** Photographer. **Trigger:** open discovery or submit filters. **Preconditions:** a searchable lab catalogue exists; personalization is optional. **Success:** the selected lab and service remain available for the booking preview. **Failure guarantee:** searching does not create an order.

1. Enter location, film format, budget or turnaround preferences.
2. Validate filters and retrieve matching active lab services.
3. Rank eligible results; optional AI recommendations explain their basis.
4. Inspect price, supported formats, address and estimated turnaround.
5. Select a lab and continue with its current service catalogue.

**Exceptions:** no matches preserves filters and suggests changing them; missing history uses ordinary ranking; unavailable AI leaves normal search available. A removed service must be revalidated before booking. **Acceptance:** a keyword and film-format filter apply together; zero results show an explicit empty state; choosing a result opens the correct lab. **Trace:** US-01, SCRUM-5/28, `apps/mobile/lib/screens/home_screen.dart`. Mobile filtering uses sample data; AI ranking is a separate Python module.

### 1.4.3 UC-02: Book Service and Arrange Collection

**Primary actor:** Photographer. **Supporting actors:** Lab and delivery partner. **Preconditions:** signed-in customer, active service at the selected lab, positive integer quantity. **Success:** an order ID, price snapshot and collection choice are recorded. **Failure guarantee:** invalid inputs do not create an order or charge the customer.

1. Select service, film details, scan options and quantity.
2. Choose direct drop-off or supply pickup address and time.
3. Calculate the estimate and show service and collection charges separately.
4. Confirm; the API checks ownership, availability and authoritative price.
5. Persist the order and request collection if required.
6. Display order code; the lab accepts or rejects capacity-dependent work.

**Exceptions:** unavailable service requires reselection; unavailable pickup offers another slot/drop-off; missing fields remain editable; a repeated request must not create duplicate orders in the target integration. **Acceptance:** quantity zero, fractional or above 100 is rejected by the current API; a client-supplied total cannot override the catalogue price. **Trace:** US-02, BR-02, SCRUM-6/28. Mobile is a preview; the independent API implements basic creation without pickup, payment or idempotency.

#### 1.4.4 UC-03: Manage the Processing Lifecycle

**Primary actor:** Lab employee. **Preconditions:** authenticated staff assigned to the order's lab; received film matches the request. **Success:** a permitted stage change records actor, time and notes. **Failure guarantee:** an invalid transition does not alter the last valid state.

1. Find the order and inspect requested service, rolls and handling notes.
2. Verify the physical receipt and record discrepancies before processing.
3. Advance through development, washing/drying, scanning and preparation.
4. Inspect quality; record a rework or delay when needed.
5. Save the stage history and notify the customer of relevant changes.

**Exceptions:** damaged film pauses the target workflow; failed quality requires a recorded rework; a stale concurrent update must be rejected rather than overwrite a newer stage. Offline retention and resynchronization are target requirements, not current web behavior. **Acceptance:** only a permitted next stage is offered; completed orders cannot advance; a scan order at quality check cannot complete without reviewed publication. **Trace:** US-03/04, BR-03, SCRUM-8/29, web domain tests. The demo combines physical substeps and persists browser state only.

#### 1.4.5 UC-04: Deliver Scans and Manage the Archive

**Primary actor:** Lab employee, followed by photographer. **Preconditions:** scan order at quality check, files ready for review and authorized customer ownership. **Success:** reviewed scans are available privately to the correct customer. **Failure guarantee:** rejected files are not published; pending transfers do not mark delivery complete.

1. Select the order and scan files; validate type, size and duplicate names.
2. Transfer bytes to private storage and verify successful transfer.
3. Review image quality and available film/camera/scan metadata.
4. Confirm publication; save metadata and append a release audit record.
5. Notify the customer, who can view, download, tag and organize the images.

**Exceptions:** invalid/corrupt files require replacement; transfer failures remain pending; unauthorized reads are denied; optional missing metadata can be added later without changing the original scan bytes. **Acceptance:** the local prototype accepts 1–100 JPEG/TIFF files, each greater than zero and at most 50 MiB; review and release confirmations are required; duplicate publication is rejected. Filename/MIME validation is not file-content inspection. **Trace:** US-05/06, BR-04/05, SCRUM-9/29/31. Only private metadata is saved locally; cloud transfer/download remains planned.

### 1.4.6 UC-05: Buy, Sell or Exchange Equipment

**Primary actors:** Seller and buyer. **Supporting actor:** Moderator. **Preconditions:** signed-in users and a valid active listing. **Success:** a transaction and final listing state are recorded. **Failure guarantee:** an unavailable listing cannot open a new transaction.

1. Seller enters category, condition, description, price and photographs.
2. Validate and publish the listing; buyers search and inspect its details.
3. Buyer contacts the seller and requests purchase or exchange.
4. Seller confirms; the system records reservation and transaction status.
5. Record fulfillment or cancellation, then allow eligible feedback.

**Exceptions:** incomplete listings stay draft; sold/hidden/expired items reject requests; disputes preserve evidence and reach the moderation queue; simultaneous buyers require server-side reservation consistency. **Acceptance:** visibility and availability have explicit labels; a moderation decision requires a reason, updates the linked listing when applicable and records an audit event. **Trace:** US-07/08, BR-06/07, SCRUM-10/30/31. The prototype supports catalogue and local moderation; payment, messaging, reservation and transaction fulfillment are not implemented end to end.

### 1.4.7 UC-06: Share Knowledge and Join Events

**Primary actors:** Expert and community member. **Supporting actor:** Moderator. **Preconditions:** contributor is permitted to publish; event dates and capacity are valid. **Success:** content or registration is stored and visible to the appropriate audience. **Failure guarantee:** rejected content is not silently published; a full event does not overbook.

1. Author submits an article, tutorial, review or event description.
2. Validate required content and perform the applicable moderation review.
3. Publish accepted content so members can search, read and discuss it.
4. Register interested members while checking capacity and duplicate entries.
5. Send event changes/reminders and preserve useful follow-up discussion.

**Exceptions:** reported content enters review; full events close or waitlist; cancellation notifies registrants; duplicate registration returns the existing registration rather than consuming another seat. **Acceptance:** date range is valid, moderation is traceable and capacity cannot be exceeded under concurrent registration. **Trace:** Core Flow 6, BR-08, SCRUM-10. The backend has basic course creation/date validation; article, discussion, enrollment and notification services remain target work.

### 1.4.8 Shared Business Rules

| Rule  | Required invariant                                                                       |
|-------|------------------------------------------------------------------------------------------|
| BR-01 | Access is scoped by role and resource ownership; a portal switch is not authentication.  |
| BR-02 | Booking quantity is a positive integer; server catalogue determines the persisted price. |

| Rule  | Required invariant                                                                             |
|-------|------------------------------------------------------------------------------------------------|
| BR-03 | Processing follows permitted transitions; the history retains actor, time and exception notes. |
| BR-04 | Scans belong to the selected order/customer and require quality review before release.         |
| BR-05 | New delivery is private; personal organization does not overwrite original scan files.         |
| BR-06 | Unavailable marketplace listings cannot accept new transactions.                               |
| BR-07 | Administrative decisions require evidence, appropriate reasons and an audit record.            |
| BR-08 | Events respect capacity; changes and cancellation reach registered members.                    |
| BR-09 | AI discloses insufficient evidence; ordinary search remains available independently.           |
| BR-10 | Demo data and proposed targets are never presented as measured production results.             |

## 1.5 Technical Requirements

This section defines the non-functional and technical requirements of the AI-powered Film Photography Platform. These requirements focus on security, performance, reliability, scalability, maintainability, data consistency, and system integration to ensure that the proposed platform can operate effectively for photographers, Film Labs, photography experts, delivery partners, administrators, and AI services.

### 1.5.1 Security and Access Control

The platform shall provide secure authentication and authorization mechanisms for all users and system components.

- The system shall support secure user authentication using JSON Web Token (JWT) and may support OAuth2-based external identity integration.
- Role-Based Access Control (RBAC) shall be used to manage permissions for different roles, including Photographer, Film Lab Owner, Photography Expert, Delivery Partner, and System Administrator. AI services require separate service identities and scoped access.
- Users shall only be allowed to access functions and information permitted by their assigned roles.
- Passwords and other sensitive credentials shall not be stored in plain text.
- Personal information, order information, payment-related information, and digital film scan files shall be protected against unauthorized access.
- Communication between client applications, backend services, and external APIs shall use secure communication protocols such as HTTPS.

### 1.5.2 Data Storage and Protection

The system shall provide reliable and consistent storage for both structured business data and digital media files.

- PostgreSQL shall be used to store structured data such as user accounts, Film Lab profiles, services, orders, reviews, and other system information.
- Digital film scans and other large media files shall be stored using cloud storage such as Azure Blob Storage or Firebase Storage.
- The system shall provide backup and recovery mechanisms to reduce the risk of data loss.
- Access to stored digital scans shall be controlled according to user permissions and ownership of the corresponding orders or resources.
- Data shall remain consistent when multiple users or services access and modify the system simultaneously.

### 1.5.3 Performance

The platform shall provide responsive services for common user and Film Lab operations under normal operating conditions.

- Common operations such as searching Film Labs, viewing services, checking order status, and accessing user information shall provide responsive results under normal operating conditions.
- Database queries shall be designed efficiently to support frequent searches, transactions, and order-related operations.
- The system shall support efficient upload, storage, and retrieval of digital film scan files.
- Caching and search optimization may be applied to improve response time for frequently requested information.

### 1.5.4 Availability and Reliability

The system shall remain reliable during normal operation and shall minimize service interruptions.

- The platform shall provide high availability for core services used by photographers and Film Labs.
- The system shall handle failures of individual services without unnecessarily affecting unrelated functions.
- Backup and recovery procedures shall be available for important application and database data.
- Order and processing information shall be stored consistently so that users can accurately track the status of their film orders.

### 1.5.5 Scalability

The architecture shall support future growth of the platform.

- The system shall be able to accommodate an increasing number of photographers, Film Labs, orders, digital images, and community activities.
- Multiple Film Labs shall be able to operate simultaneously on the same platform.
- Backend services shall be designed so that computing resources can be increased when system traffic grows.
- Cloud infrastructure shall support future expansion of storage, processing, and application services.

### 1.5.6 Modularity and Maintainability

The platform shall use a modular architecture to simplify development, testing, maintenance, and future upgrades.

- Major components shall be separated into Mobile Application, Web Portal, Backend Services, AI Services, Database, Search Engine, and Cloud Storage.
- Components shall communicate through well-defined APIs and data interfaces.
- Changes to one component shall have minimal impact on unrelated components.
- AI services and other external integrations shall be designed so that they can be replaced or upgraded without requiring a complete redesign of the platform.

### 1.5.7 API and System Integration

The platform shall provide integration capabilities for internal and external services.

- RESTful APIs shall be used for communication between the mobile application, web portal, back-end services, and other system components.
- APIs shall support integration with payment gateways, logistics providers, cloud storage services, and AI services.
- Data exchanged through APIs shall follow consistent data formats and secure communication mechanisms.
- The architecture shall allow additional photography-related services to be integrated in the future.

### 1.5.8 Search and Data Access

The platform shall provide efficient access to Film Lab and photography-related information.

- A search engine such as Elasticsearch or OpenSearch may be used for Film Lab, service, marketplace, and photography knowledge searches.
- Search results should be relevant, responsive, and filterable according to available system attributes.
- Authorized users shall be able to retrieve their own orders, digital scans, and related information efficiently.

### 1.5.9 Deployment and Continuous Integration

The system shall support modern cloud deployment and software development practices.

- Microsoft Azure may be used as the main cloud deployment platform.
- GitHub shall be used for source-code version control and team collaboration.
- GitHub Actions may be used to automate build, testing, and continuous integration and deployment (CI/CD) activities.
- Deployment configurations shall be separated from application source code where appropriate to improve security and maintainability.

## 1.6 Theory and Practical

The proposed platform combines software engineering principles, cloud computing, database management, artificial intelligence, search technologies, and mobile and web development. This section describes the theoretical foundations and the practical technologies proposed for implementing the system.

### 1.6.1 Software Engineering Architecture

The platform follows a modular and service-oriented approach in which major system components are separated according to their responsibilities.

The main components include the Mobile Application, Web Portal, Backend Services, AI Services, Database, Search Engine, and Cloud Storage. This separation improves maintainability because each component can be developed, tested, and upgraded independently.

The architecture also uses API-based communication between clients and backend services. This approach allows the mobile and web applications to share common business logic while supporting future integrations.

### 1.6.2 Mobile Application – Flutter

Flutter is proposed for developing the mobile application for Android and iOS.

The mobile application is designed primarily for photographers and community users. It can provide functions such as:

- searching and discovering Film Labs;
- viewing film processing services and prices;
- booking film processing services;
- tracking film processing and delivery status;
- receiving and accessing digital film scans;
- managing the Digital Film Archive;
- participating in the photography community and marketplace; and
- interacting with the AI Photography Assistant.

Using Flutter allows the project to develop a cross-platform mobile application from a shared codebase.

### 1.6.3 Web Portal – ReactJS / Next.js

ReactJS or Next.js is proposed for the web-based interface.

The web portal is mainly intended for Film Lab owners and administrators. It can support Film Labs in managing their profiles, service packages, customer orders, processing workflows, digital scan delivery, customer information, and operational reports.

The web portal can also provide administration functions for managing users, Film Labs, services, community content, and other platform resources.

### 1.6.4 Backend Services – ASP.NET Core Web API / Node.js

ASP.NET Core Web API or Node.js with Express can be used to implement the backend layer.

The backend is responsible for:

- implementing business logic;
- receiving and processing requests from mobile and web applications;
- communicating with the PostgreSQL database;
- providing RESTful APIs;
- managing authentication and authorization;
- processing order and service workflows; and
- integrating external services such as AI, payment, logistics, and cloud storage.

This layer acts as the central connection between the client applications and the platform's data and services.

### 1.6.5 Database – PostgreSQL

PostgreSQL is proposed as the relational database management system.

The database stores structured information including:

- user accounts and roles;
- photographer and Film Lab profiles;
- film processing services and service packages;
- orders and processing status;
- reviews and ratings;
- marketplace information; and
- other operational and community data.

A relational database is appropriate for maintaining relationships and consistency among users, Film Labs, services, orders, and other business entities.

### 1.6.6 Authentication – JWT and OAuth2

JWT and OAuth2 are proposed to support secure authentication and authorization.

JWT can be used to maintain authenticated access to protected backend APIs. After successful authentication, a token can be used to identify the user when accessing authorized resources.

OAuth2 can support authentication through external identity providers when required. Authentication can be combined with Role-Based Access Control (RBAC) so that each user receives permissions according to their role.

### 1.6.7 Cloud Storage – Azure Blob Storage / Firebase Storage

Azure Blob Storage or Firebase Storage can be used to store digital film scans and other media resources. Digital scans can be significantly larger than normal structured database records. Therefore, storing these files in dedicated cloud storage can reduce the burden on the relational database while providing scalable storage.

The storage layer should support secure access, backup, recovery, and future growth of the Digital Film Archive.

### 1.6.8 Artificial Intelligence Integration

Artificial Intelligence is one of the main technologies that differentiates the proposed platform from a basic Film Lab booking system.

The proposed AI components include:

- **OpenAI GPT API:** provides conversational AI capabilities for the AI Photography Assistant.
- **LangChain or Semantic Kernel:** supports the development and orchestration of AI-powered workflows.
- **Retrieval-Augmented Generation (RAG):** allows the AI Assistant to retrieve domain-specific photography knowledge before generating an answer.
- **Recommendation Systems:** can recommend suitable Film Labs, film types, processing services, and learning resources according to user needs.
- **Computer Vision:** can be used as a proposed technology for analyzing the quality or characteristics of scanned film images.

These AI technologies support personalized assistance and intelligent recommendation rather than replacing the core Film Lab processing workflow.

### 1.6.9 Search Engine – Elasticsearch / OpenSearch

Elasticsearch or OpenSearch can be used to provide efficient search capabilities.

The search engine can support:

- Film Lab discovery;
- service and film-type searching;
- marketplace searching; and
- retrieval of photography-related knowledge and community content.

Search technology is especially useful when the number of Film Labs, services, products, and knowledge resources increases.

### 1.6.10 Cloud Deployment – Microsoft Azure

Microsoft Azure is proposed as the cloud deployment platform.

Azure can provide infrastructure for deploying backend services, databases, cloud storage, and other components. Cloud deployment supports scalability because computing and storage resources can be expanded as the number of users, orders, and digital images grows.

### **1.6.11 CI/CD – GitHub Actions**

GitHub Actions can be used to support Continuous Integration and Continuous Deployment (CI/CD).

The development workflow can automatically build and test project components when changes are pushed to the repository. After successful validation, the workflow can prepare or deploy updated components according to the project's deployment configuration.

This reduces repetitive manual tasks and helps maintain consistency during team development.

### **1.6.12 Version Control and Collaboration – GitHub**

GitHub is used as the version control and collaboration platform for the project.

Team members can work on separate branches, record changes through commits, review changes through pull requests, and merge completed work into the main branch. This workflow is particularly useful for a multi-member software engineering project because it allows different members to develop different sections or components while maintaining a shared project history.

### **1.6.13 Technology Integration**

The technologies above are integrated into a single platform architecture.

The Mobile Application and Web Portal communicate with the Backend Services through RESTful APIs. The Backend communicates with PostgreSQL for structured data and cloud storage for digital film scans. Search services support efficient discovery of Film Labs and content. AI services provide personalized assistance and recommendations. Azure provides cloud deployment, while GitHub and GitHub Actions support version control and CI/CD.

Together, these technologies provide the theoretical and practical foundation for building a scalable digital ecosystem connecting photographers, Film Labs, photography experts, delivery partners, administrators, and AI-powered services.

## **1.7 Expected Deliverables**

The project is expected to deliver a cloud-deployable Minimum Viable Product (MVP) that demonstrates the complete analog photography service lifecycle, from Film Lab discovery and service booking to processing tracking, digital delivery, long-term archiving, and community engagement. The expected products are described below.

### **Ứng dụng di động dành cho nhiếp ảnh gia**

A cross-platform Flutter application for Android and iOS will provide the main customer-facing experience. Photographers will be able to register and manage their accounts, discover and compare Film Labs, configure and book services, request pickup and delivery, complete online payments, track order progress, receive notifications, access digital scans, manage their personal archive, participate in community features and the Marketplace, and interact with the AI-powered Photography Assistant.

## **Cổng thông tin web quản lý cơ sở xử lý phim**

A ReactJS or Next.js portal will allow Film Lab owners and operators to manage laboratory profiles, supported film formats, service packages, prices, processing capacity, and expected turnaround times. It will also support order verification, workflow scheduling, processing-stage updates, digital scan delivery, customer management, revenue monitoring, and operational reporting.

## **Hệ thống quản trị web**

A centralized administration application will support the management of users, roles, permissions, Film Lab approvals, service categories, transaction fees, payments, Marketplace operations, reviews, community content, complaints, and disputes. Audit information and platform-level monitoring will help administrators maintain safe and consistent operations.

## **Mô-đun sàn giao dịch**

The Marketplace will allow community members to publish, search for, buy, sell, or exchange analog cameras, lenses, film rolls, and photography accessories. Its scope will include listing management, search and filtering, seller profiles, ratings, transaction tracking, and communication between buyers and sellers.

## **Nền tảng cộng đồng và chia sẻ kiến thức**

Photography Experts and experienced community members will be able to publish tutorials, technical articles, equipment reviews, Film Lab reviews, and practical experiences. The platform will also support discussions, questions and answers, workshops, photowalks, and event registration, thereby creating a continuously expanding photography knowledge base.

## **Hệ thống quản lý kho lưu trữ phim kỹ thuật số**

A secure cloud-based Digital Film Archive will allow photographers to preview, download, classify, search, and organize scanned photographs. Each digital asset may retain metadata such as film stock, camera model, lens, shooting date, Film Lab, processing method, and scanning specifications. Access control, storage policies, backup, and recovery mechanisms will protect users' collections.

## **Hệ thống đề xuất AI**

The Recommendation Engine will use user preferences, service history, geographical constraints, Film Lab capabilities, prices, turnaround times, and community feedback to rank suitable Film Labs and services. It will also support personalized film-stock and educational-resource suggestions. The deliverable will include a callable AI service, a documented evaluation method, and comparative results for the selected recommendation approach.

## **Trợ lý nhiếp ảnh ứng dụng AI**

The Photography Assistant will combine a Large Language Model with Retrieval-Augmented Generation and a curated domain knowledge base to answer photography-related questions with relevant context. The AI package will also include semantic retrieval for knowledge and service discovery and a Computer

Vision prototype for detecting common scan-quality problems such as blur, exposure errors, color casts, dust, and scratches. Evaluation results, limitations, and responsible-use guidance will accompany these components.

### **Dịch vụ phía máy chủ và hạ tầng đám mây**

A modular backend will provide RESTful APIs, PostgreSQL persistence, JWT and OAuth2 authentication, Role-Based Access Control, multi-tenant data isolation, cloud file storage, search indexing, and AI-service orchestration. Integration interfaces will support payment gateways and third-party logistics providers. The cloud infrastructure will include CI/CD, logging, monitoring, backup, and recovery mechanisms required to deploy and operate the MVP.

### **Minh chứng bàn giao**

The final handover will include source code, database migrations, API specifications, architecture and database diagrams, test plans and results, AI experiment reports, deployment instructions, user guides, technical documentation, the final Capstone report, presentation slides, and a product demonstration. These artifacts will provide verifiable evidence that the products above satisfy the agreed functional and non-functional requirements.

## **1.8 Proposed Tasks and Responsibilities**

To distribute the workload fairly, the project is organized into five comparable work packages. Each member owns the analysis and design activities, implementation or research, testing, and documentation associated with the assigned package. The planned team capacity is allocated at approximately 20% per member and will be monitored through sprint backlog items and story points.

### **Gói công việc 1: Bối cảnh, giải pháp và yêu cầu nghiệp vụ**

**Người phụ trách: Nguyễn Thái Gia Bảo**

- Analyze the project context, current problems, stakeholder needs, and the proposed value of the unified film-photography ecosystem.
- Define system actors, functional requirements, business rules, shared terminology, acceptance criteria, and the six core business flows.
- Prepare user stories and trace them to the relevant actors, use cases, and business processes.
- Produce the Use Case, Business Flow, and Activity diagrams and document the assumptions and boundaries used by the remaining work packages.

The expected outputs are the context and problem analysis, proposed solution, actor catalogue, requirements baseline, business rules, user stories, Use Case Diagram, and six documented Business Flows.

### **Gói công việc 2: Yêu cầu phi chức năng, cơ sở dữ liệu và kiến trúc**

**Người phụ trách: Lê Thị Như Quân**

- Define measurable non-functional requirements for security, performance, availability, scalability, data consistency, maintainability, backup, and recovery.
- Select and justify the technology stack and design the modular, multi-tenant system architecture and storage architecture.
- Design the PostgreSQL schema, entity relationships, constraints, indexes, data-isolation rules, models, and migration or SQL scripts.
- Produce the System Architecture, Entity–Relationship, Database Architecture, and Storage Architecture diagrams and verify their consistency.

The expected outputs are the non-functional requirements, technology rationale, system and storage architecture, ERD, database schema, models, migrations or SQL scripts, and supporting architecture documentation.

### **Gói công việc 3: Hệ thống phía máy chủ và API**

**Người phụ trách: Huỳnh Gia Hợp**

- Define RESTful API contracts and implement the backend with ASP.NET Core or Node.js, including validation, error handling, logging, and persistence.
- Implement JWT/OAuth2 authentication, Role-Based Access Control, user and authorization flows, and tenant-aware business services.
- Implement order and Film Lab management, processing lifecycle, payment, digital delivery, archive metadata, and required integration contracts.
- Produce the API Architecture, Sequence, and API Flow diagrams together with API documentation, unit tests, and backend integration tests.

The expected outputs are the backend services, documented REST API, authentication and authorization, core business logic, payment/order/Lab management, API diagrams, and backend test evidence.

### **Gói công việc 4: Ứng dụng di động và cổng thông tin web**

**Người phụ trách: Nguyễn Đào Quốc Khánh**

- Design the UI/UX and implement the Flutter photographer application and responsive ReactJS or Next.js web portals.
- Implement the Photographer App flows, Film Lab management portal, System Administrator portal, Marketplace UI, and Digital Archive UI.
- Integrate screens with the documented backend APIs and handle loading, empty, validation, authorization, offline, and error states consistently.
- Produce the UI Flow, Component/Frontend Architecture, and relevant Sequence diagrams; perform usability and frontend integration tests.

The expected outputs are the Flutter application, ReactJS/Next.js web portals, reusable UI components, responsive interfaces, UI and frontend diagrams, API integrations, user-flow documentation, and frontend test evidence.

## Gói công việc 5: AI, kiểm thử và triển khai

**Người phụ trách:** Nguyễn Thành Toàn

- Prepare datasets, a curated photography knowledge base, baselines, and evaluation protocols for Film Lab recommendation, RAG, semantic search, the AI Assistant, and Computer Vision-based scan-quality analysis.
- Implement documented AI services for recommendation, retrieval and grounded assistance, and detection of blur, exposure problems, color casts, dust, and scratches; report limitations and responsible-use considerations.
- Define and execute unit, contract, integration, end-to-end, security, performance, recovery, and AI evaluation tests; record measurable pass/fail criteria and trace defects to closure with the relevant component owner.
- Produce the AI Architecture and AI Workflow diagrams; configure Docker, GitHub Actions, Azure/cloud deployment, CI/CD, health checks, logging, monitoring, secrets handling, backup verification, and rollback procedures.
- Produce the Deployment Diagram and consolidate AI experiment evidence, test results, deployment instructions, and the AI–Testing–Deployment presentation and demonstration package.

The expected outputs are the AI service prototypes, recommendation engine, versioned knowledge base, semantic retriever, grounded Photography Assistant, scan-quality analyzer, three required diagrams, automated tests and evaluation reports, CI/CD pipeline, cloud deployment artefacts, operations guide, and the AI–Testing–Deployment presentation package.

## Trách nhiệm chung và kiểm soát khối lượng công việc

All members participate in sprint planning, peer review, integration testing, issue resolution, and the final demonstration. Each member remains responsible for correcting defects and supplying documentation for the owned component. Toàn owns the AI, system-level testing, and deployment package, while the owners of requirements, architecture/database, backend/API, and mobile/web remain responsible for fixing defects in their components. Backlog estimates and actual effort will be reviewed at the end of each sprint; work will be redistributed when a package materially exceeds its planned 20% share.

## Trình tự triển khai

1. **Yêu cầu và kiến trúc:** establish the context, requirements baseline, system/data/storage architecture, API contracts, and acceptance criteria.
2. **Phát triển và nghiên cứu song song:** implement the backend, mobile and web interfaces, and AI prototypes against the agreed contracts.
3. **Tích hợp tăng dần:** connect mobile/web, backend, data, storage, AI, payment, and logistics components early and resolve contract mismatches.
4. **Kiểm thử hệ thống và đánh giá AI:** execute end-to-end quality assurance while evaluating AI accuracy, retrieval quality, groundedness, and scan-analysis performance.
5. **Triển khai và hoàn thiện:** deploy the MVP, verify operations, complete the documentation, and prepare the final report, presentation, and demonstration.

**Tích hợp tổng thể dự án**

The deliverables collectively form a unified ecosystem for analog photography. The mobile application is the primary customer interface, while the Film Lab and administration portals support service operations and platform governance. Backend services, cloud storage, search, and the Digital Film Archive provide a shared technical foundation. The Marketplace and knowledge-sharing modules support sustainable community engagement, and the AI layer enhances the ecosystem through personalized recommendations, semantic retrieval, contextual assistance, and scan-quality analysis.

The five work packages directly correspond to the functional requirements, non-functional requirements, research objectives, and expected products. Clear ownership makes progress measurable, while the shared review, integration, testing, and documentation responsibilities preserve collective accountability for the final system.

## Chapter 2

# System and Interface Design

### 2.1 System Architecture

The proposed platform uses a layered, modular architecture. The Flutter Photographer App and the React web portals are presentation clients. They communicate with the Backend API through authenticated HTTPS requests. The backend owns business rules and access decisions; client-side role selectors are interface demonstrations and must not be treated as authorization.

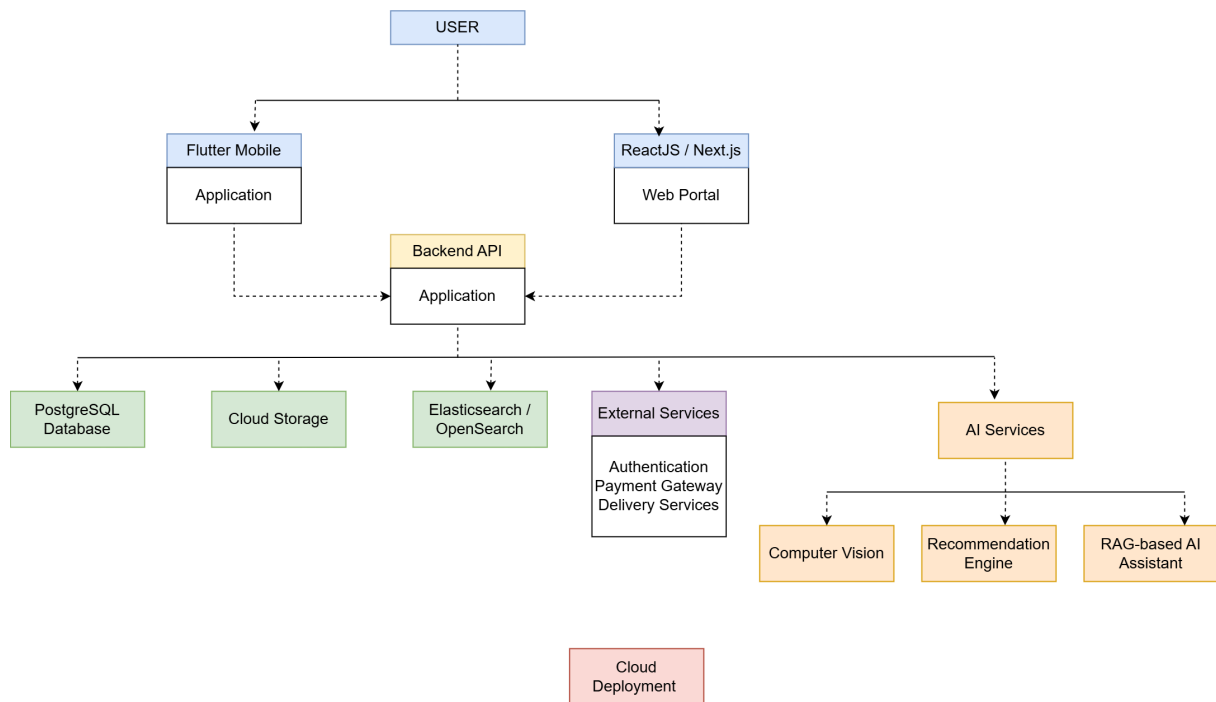


Figure 2.1: Proposed system architecture: shared API, data, storage, search, and AI services

#### 2.1.1 Component Responsibilities

##### Client applications

Present discovery, booking, tracking, archive, marketplace, and administration screens. Validate input for usability and show loading, empty, and failure states. The API repeats all security-sensitive validation independently.

**Backend API**

Authenticate users, enforce role and resource ownership, validate prices and services, manage the order lifecycle, and record audit events. PostgreSQL transactions protect related state changes.

**PostgreSQL**

Store accounts, labs, services, orders, workflow history, and media metadata. Foreign keys and indexes support consistency and searches.

**Object storage**

Store original scan files separately from relational records. Access is private by default; authorized, short-lived links provide downloads after ownership and order checks.

**AI service**

Rank labs, retrieve domain knowledge, answer grounded questions, and analyze scan quality. A backend adapter isolates this service from public clients and prevents external model keys from reaching browsers.

**Search and external adapters**

Search indexes, payment providers, and logistics providers are proposed integration boundaries. They require independent credentials, failure handling, and contract tests.

**2.1.2 Core Data Flow and Failure Handling**

A booking request contains selected lab/service identifiers and film details. The backend validates ownership and availability, derives the charge from the service catalogue, stores the order, and returns its identifier. Lab staff advance only allowed status transitions. The status event and current order state are committed together so a retry cannot silently duplicate a business event.

For delivery, staff upload a validated file, attach metadata to the correct order, and publish it after quality checking. The photographer sees only scans belonging to an authorized order. Failed or incomplete uploads stay unpublished. Payment and logistics callbacks must be authenticated and idempotent before they change order state. If the AI service is unavailable, ordinary discovery and order processing remain usable.

**2.1.3 Current Repository Boundary**

The diagram is the target design. The current repository implements clients under `apps/`, an Express service under `backend/`, and Python AI modules under `ai_service/`. The AI prototype uses local recommendation data, TF-IDF retrieval, an extractive assistant, and image-quality heuristics. It does not establish a deployed GPT integration or trained computer-vision model. The frontend prototypes remain demonstrations until shared backend, identity, storage, and notification contracts are connected and tested.

**2.2 Technology Stack**

This section summarizes the proposed technology stack for the AI-powered Film Photography Platform. The selected technologies support the development of the mobile application, web portal, backend services, database, cloud storage, artificial intelligence, search engine, deployment, and software development workflow.

Table 2.1: Proposed Technology Stack

| Category            | Technology                               | Role and Technical Justification                                                                                                                                                      |
|---------------------|------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Mobile Application  | Flutter                                  | Cross-platform framework for developing the mobile application for Android and iOS from a shared codebase. It supports the main photographer and community functions of the platform. |
| Web Portal          | ReactJS / Next.js                        | Used to develop the web portal for Film Lab owners and administrators. It supports service management, order management, customer management, reports, and administration functions.  |
| Backend Services    | ASP.NET Core Web API / Node.js (Express) | Provides business logic, RESTful APIs, authentication, authorization, database communication, and integration with external services.                                                 |
| Database            | PostgreSQL                               | Relational database for storing structured data such as users, Film Labs, services, orders, processing status, reviews, marketplace information, and community data.                  |
| Authentication      | JWT / OAuth2                             | Provides secure authentication and authorization. JWT can be used for authenticated API access, while OAuth2 can support external authentication and authorization flows.             |
| Cloud Storage       | Azure Blob Storage / Firebase Storage    | Stores digital film scans and other media resources. Cloud storage supports secure access, backup, recovery, and scalability.                                                         |
| AI Integration      | OpenAI GPT API                           | Provides conversational AI capabilities for the AI-powered Photography Assistant.                                                                                                     |
| AI Framework        | LangChain / Semantic Kernel              | Supports the development and orchestration of AI-powered workflows and integration with external knowledge and services.                                                              |
| Knowledge Retrieval | Retrieval-Augmented Generation (RAG)     | Allows the AI Assistant to retrieve relevant photography knowledge and provide domain-specific responses.                                                                             |
| Recommendation      | Recommendation System                    | Provides personalized recommendations for Film Labs, services, film types, and educational resources based on user preferences and available platform information.                    |
| Computer Vision     | Computer Vision Models                   | Supports analysis of scanned film images and can be used for scan-quality evaluation.                                                                                                 |
| Search Engine       | Elasticsearch / OpenSearch               | Provides efficient search for Film Labs, services, marketplace items, and photography-related knowledge and content.                                                                  |
| Cloud Deployment    | Microsoft Azure                          | Provides cloud infrastructure for deploying backend services, storage, databases, and other platform components while supporting scalability.                                         |
| CI/CD               | GitHub Actions                           | Automates software build, testing, and deployment workflows.                                                                                                                          |
| Version Control     | GitHub                                   | Provides source-code version control, collaboration, change tracking, and project repository management.                                                                              |

## Technical Dependencies and Limitations

The mobile and web applications depend on the Backend API for business logic and data access. The backend depends on PostgreSQL for structured data storage and on cloud storage for digital film scans and other media files.

AI features depend on external AI services and supporting technologies such as the OpenAI GPT API, LangChain or Semantic Kernel, RAG, recommendation algorithms, and computer vision models. Search-related functions depend on Elasticsearch or OpenSearch.

The final choice between alternative technologies such as ASP.NET Core Web API and Node.js, ReactJS and Next.js, or Azure Blob Storage and Firebase Storage should be confirmed during implementation according to project requirements, team expertise, integration needs, and deployment constraints.

### 2.2.1 Database and Digital Storage Architecture

This section defines the proposed database and digital storage architecture for the AI-powered Film Photography Platform.

#### Database Architecture

PostgreSQL is proposed as the primary relational database for storing structured system data. The database stores information related to users, films, Film Labs, services, orders, reviews, community activities, and film scan metadata.

The Backend API acts as the main access layer between the mobile/web applications and PostgreSQL. Client applications send API requests to the Backend API, which handles business logic, authentication, authorization, and database access.

The main data entities include:

- User
- Film
- Film Lab
- Service
- Order
- Review
- Community
- Film Scan Metadata

#### Digital Storage Architecture

Digital film scans are separated from structured relational data. PostgreSQL stores scan metadata such as file name, file type, file size, file path, and related information, while the actual film scan files are stored in cloud storage.

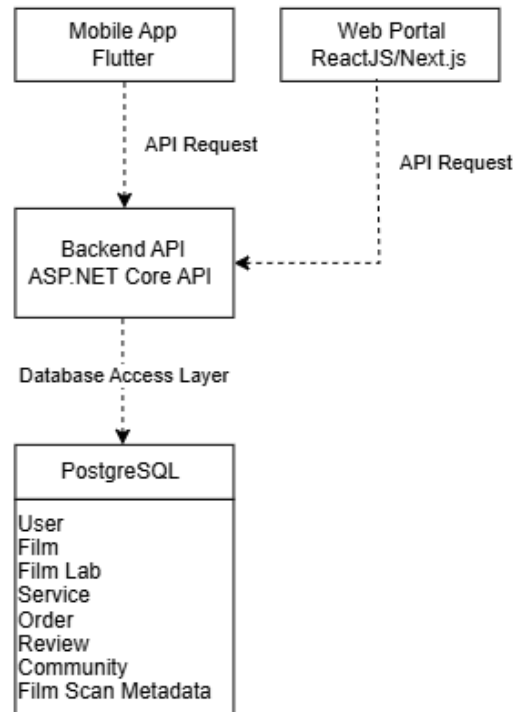
The proposed architecture uses the Backend API as the access layer for uploading and retrieving film scans. This approach separates structured data management from large media file storage and supports scalability and secure file access.

**Data Access and Security**

The Backend API controls access to both PostgreSQL and cloud storage. Authentication and authorization mechanisms are applied before users can access protected data or digital film scans.

This architecture provides centralized data access, supports backup and recovery, and allows the storage system to scale as the number of users and film scans increases.

## Database Architecture Diagram



## Digital Storage Diagram

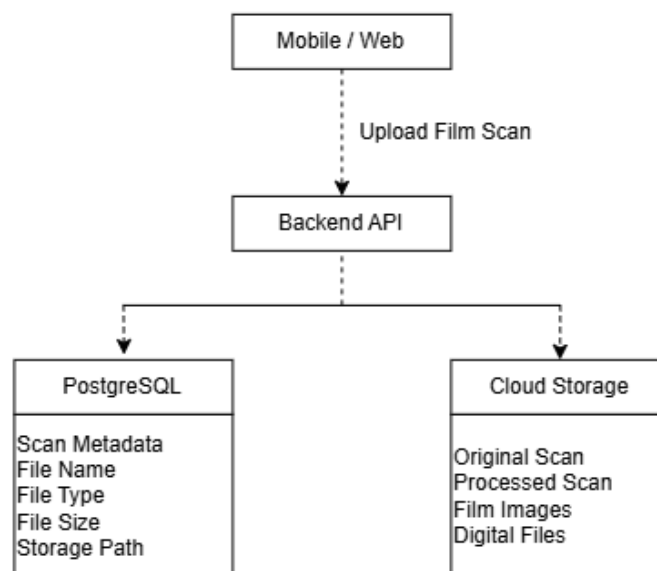


Figure 2.2: Database and Digital Storage Architecture

## 2.3 Implemented Data Model and API Contracts

The earlier database/storage diagram describes the target platform. This section documents the smaller executable baseline in `backend/schema.sql` and `backend/server.js`. It prevents planned entities from being mistaken for tables already available in the submission.

### 2.3.1 Current Entity Relationships

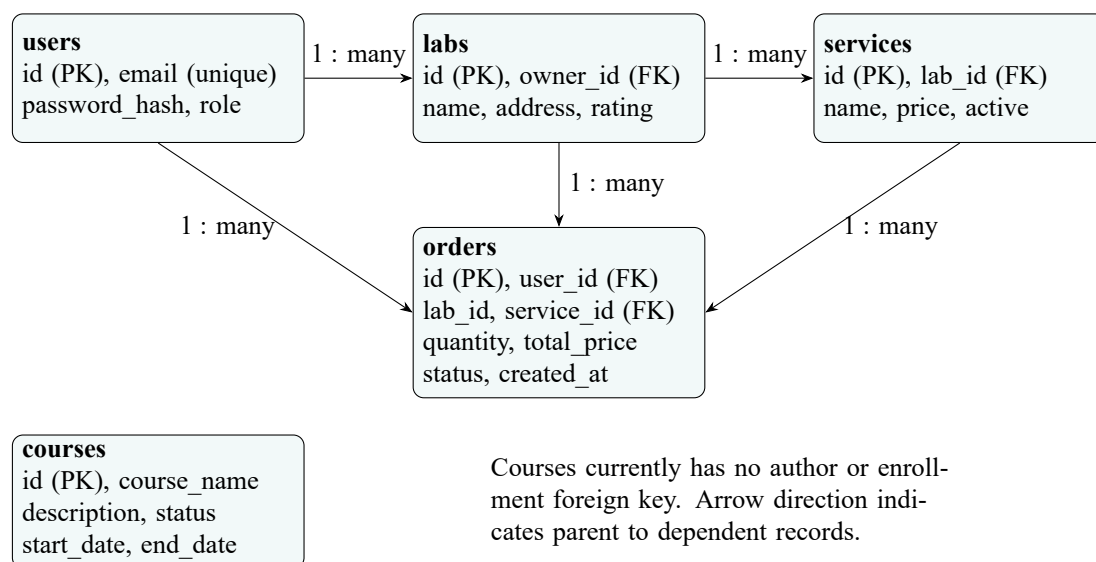


Figure 2.3: ERD derived from the current five-table bootstrap schema

### 2.3.2 Constraints and Data Dictionary

| Entity / field | Meaning and implemented constraint                                                                                                                                                                                                          |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| users          | Unique, non-null email; non-null password hash. Roles: photographer, lab_owner, expert, moderator, admin. Employee/delivery identities are not yet implemented.                                                                             |
| labs           | owner_id references users; name/address required; rating is between 0 and 5 and starts at 0. API role validation controls creation, while SQL only checks the owner exists.                                                                 |
| services       | Belongs to a lab; positive integer-valued NUMERIC price; active flag controls availability for order creation. Price is interpreted as VND by the current application.                                                                      |
| orders         | User, lab and service references required; quantity 1–100; positive total; timestamp defaults to database time. API verifies service/lab association. Separate foreign keys alone do not enforce that association for arbitrary SQL writes. |
| order status   | Database text defaults to pending; no SQL transition constraint. The browser demo has its own six-state enum; it is not synchronized with this field.                                                                                       |
| courses        | Status limited to draft, published or cancelled; end date cannot precede start date. No participant or capacity table exists yet.                                                                                                           |

| Entity / field | Meaning and implemented constraint                                                                                                     |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Indexes        | orders(user_id) supports ownership filtering; services(lab_id) supports service lookup. These are definitions, not benchmark evidence. |

The bootstrap is for a new local database. Existing team databases require reviewed migrations and backup. Scan, payment, pickup, moderation, event registration and server audit tables remain additions to the target design.

### 2.3.3 Current HTTP Contract

Development base URL: `http://127.0.0.1:3000`. Protected calls use `Authorization: Bearer <token>`. Login returns a one-hour HS256 token; both signing and verification read the same secret of at least 32 characters.

| Endpoint                    | Access / input                                                       | Successful response                                                                                            |
|-----------------------------|----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| POST /auth/login            | email, password; public                                              | 200: message, token, user(id,email,role).                                                                      |
| GET /labs; /courses         | Any valid user token                                                 | 200: database row array. Course listing currently includes all statuses; publication filtering is future work. |
| GET /orders                 | Any valid token                                                      | 200: own orders; admin can list all. This is not a lab-staff assignment query.                                 |
| POST /labs                  | lab_owner/admin; name, address, optional description                 | 200: created row, owner from token and rating 0.                                                               |
| POST /courses               | expert/admin; course_name, description, status, start_date, end_date | 200: created course row.                                                                                       |
| POST /orders                | photographer/admin; integer lab_id, service_id, quantity             | 200: created pending order; total from active service price times quantity.                                    |
| GET /users; /users/full     | admin                                                                | 200: id, email, role only; /users/full limits results to five.                                                 |
| GET /tables; /schema/:table | admin                                                                | 200: table or column metadata; intended for development inspection.                                            |

Malformed JSON or invalid fields return 400; invalid/missing credentials return 401; an authenticated but disallowed role returns 403. Database failures return a generic 500 message. Existing creation routes return 200, not 201; documentation records the actual implementation rather than assuming a different contract.

#### Order example (illustrative, requires seeded IDs):

```
POST /orders
Authorization: Bearer <token>
```

Content-Type: application/json

```
{"lab_id": 1, "service_id": 2, "quantity": 3}
```

If the selected service costs 120,000 VND, the persisted total is 360,000 VND. The example is arithmetic and a contract illustration, not a captured live database response. No endpoint accepts the browser's local storage snapshot as trusted authorization or order data.

### 2.3.4 Integration Decisions and Remaining Contracts

#### Current stack

React/TypeScript/Vite, Flutter sources, Express/PostgreSQL and a separate Python AI service are present. Next.js, ASP.NET Core, OAuth2, hosted GPT and managed search listed earlier are candidate technologies, not additional installed implementations.

#### Private scan design

Target storage keeps file bytes separate from SQL metadata. An authorized API should issue short-lived access, verify upload completion and publish metadata transactionally. The current demo stores only file name, size and type; it does not implement this storage protocol.

#### Status integration

Agree a shared order enum, allowed transition table, resource ownership checks and concurrency/version behavior before connecting the frontend. Preserve physical processing detail in history even if the UI groups stages into simpler labels.

#### Transaction boundaries

Reservation, pickup, payment and notification need idempotency and failure recovery contracts. A browser success message is not evidence of a committed external transaction.

## 2.4 Mobile and Web Design

This section presents the proposed user interfaces for the Photographer Mobile Application, Film Lab Web Portal, Administration Web Portal, Marketplace, and Digital Film Archive. Flutter is proposed for the mobile application, while React with TypeScript and Vite is used for the current web prototype. The screen catalogue below describes the target interface; implemented interactions and missing integrations are identified in the handover chapter.

### 2.4.1 UI/UX Guidelines

The interfaces follow these common guidelines:

- The mobile application uses simple bottom navigation for Home, Orders, Archive, Marketplace, and Profile.
- The Film Lab and Administration portals use a consistent sidebar, page header, searchable tables, filters, and pagination.
- Each screen has one clear primary action, such as booking a service, updating an order status, or uploading scan files.
- Order states use both color and text labels. Important examples are Pending, Processing, Scanning, Ready for Delivery, and Completed.
- Forms show visible labels and short validation messages without removing valid information entered by the user.
- Loading, empty, success, confirmation, and error states are provided for important actions.
- Destructive actions and actions that affect another user require a confirmation step.
- Mobile and web layouts remain readable on their supported screen sizes and provide keyboard focus for web controls.

### 2.4.2 Photographer Mobile Application

The Photographer Application is a Flutter source prototype targeting mobile platforms; no Android or iOS device build is claimed in this snapshot. The target product allows photographers to find a Film Lab, create processing orders, track their film, receive scan files, use the Marketplace, and manage a personal Digital Film Archive.

The main booking flow is:

Login → Search Film Lab → Compare Services → Enter Film Details → Choose Pickup → Confirm Order → Track Status

Before confirmation, the application displays the service price, pickup fee, estimated completion time, and total amount. If location permission is not available, the photographer can manually enter a district or city.

Table 2.4: Main Photographer Mobile Application Screens

| Screen Group              | Main Functions                                                                                               |
|---------------------------|--------------------------------------------------------------------------------------------------------------|
| Authentication            | Register, log in, reset password, and verify an account.                                                     |
| Home                      | Show current orders, nearby Film Labs, recommendations, and notifications.                                   |
| Film Lab Discovery        | Search and filter by location, film format, service, price, rating, and processing time.                     |
| Booking                   | Select a service, enter film information, choose pickup or drop-off, review the cost, and confirm the order. |
| Orders                    | Display order history and the processing status timeline.                                                    |
| Scan Delivery             | Preview thumbnails, download scan files, and save photographs to the archive.                                |
| Marketplace and Community | View listings and community content, contact sellers, and report inappropriate content.                      |
| AI Assistant              | Ask questions about film stocks, exposure, camera settings, Film Labs, and processing services.              |
| Profile                   | Update personal details, saved addresses, notifications, and account settings.                               |

### 2.4.3 Film Lab Web Portal

The Film Lab Web Portal gives owners and employees a larger workspace for handling orders and scan files. A Lab owner can manage the Film Lab profile, services, prices, employees, and reports. A Lab employee can process assigned orders and upload scan files.

The main pages are Dashboard, Orders, Order Details, Processing Board, Schedule, Scan Delivery, Services and Pricing, Customers, Reports, and Lab Settings. The processing workflow is:

New Order → Accepted → Film Received → Developing → Scanning → Quality  
Checking → Scans Published → Completed

Every status update records the employee, update time, and an optional note. Invalid status jumps are disabled. During scan delivery, the portal validates file type and size, displays upload progress, supports retry, and requires a final confirmation before publishing files to the photographer.

### 2.4.4 Administration Web Portal

The Administration Portal supports system administrators, moderators, finance staff, and support staff. Role-based permissions limit each user to the data and actions required for their work.

The portal provides the following main functions:

- manage users, roles, account status, and Film Lab applications;
- manage service categories, platform fees, and public settings;
- monitor payments, failed transactions, refunds, and activity;
- review community content, marketplace listings, complaints, and disputes;
- view reports and search the administrative audit log.

For Film Lab approval, an administrator checks the submitted business and service information, requests

missing information when necessary, and records an approval or rejection reason. Important access, moderation, payment, and configuration actions are written to the audit log.

### 2.4.5 Marketplace Interface

The Marketplace supports cameras, lenses, film rolls, accessories, and other film photography equipment. Users can search by keyword and filter by category, brand, condition, location, price, and listing date.

The main screens are Marketplace Home, Search Results, Listing Details, Create Listing, Seller Profile, Messages, Saved Items, My Listings, and Transactions. A seller creates a listing using the following flow:

Select Category → Upload Photos → Enter Item Details → Set Price and Delivery → Preview → Submit

The interface clearly marks Draft, Active, Reserved, Sold, Hidden, Rejected, and Expired listings. Users can report suspicious listings, messages, or accounts, and the report is sent to the Administration Portal.

### 2.4.6 Digital Film Archive Interface

The Digital Film Archive stores scan files published by Film Labs and helps the photographer organize them by order, album, film stock, shooting date, tags, and favorite status.

The archive includes a library, order scan view, photo viewer, album manager, search and filters, metadata editor, downloads, and storage and privacy settings. Film Lab metadata, such as order code, scanner, resolution, and delivery date, is separated from personal metadata entered by the photographer. Changing an album, tag, or note does not overwrite the original delivered file.

New scan deliveries are private by default. Sharing requires a separate user action. Large downloads show progress and can be retried if the network is interrupted.

### 2.4.7 Frontend Component Architecture

The Flutter application and ReactJS or Next.js portals use the same separation of responsibilities.

Table 2.5: Frontend Architecture Layers

| Layer             | Responsibility                                                                                   |
|-------------------|--------------------------------------------------------------------------------------------------|
| Presentation      | Screens, pages, layouts, forms, tables, dialogs, and reusable interface components.              |
| State and Feature | Authentication, order, marketplace, archive, profile, and notification state.                    |
| Domain            | Shared models and application rules for users, Film Labs, services, orders, scans, and listings. |
| API and Service   | REST API communication, file upload and download, and real-time updates.                         |
| Shared            | Theme, routing, form validation, error mapping, localization, and common utilities.              |

A user action starts in the presentation layer. The feature state validates the input and calls the relevant service. The service communicates with the backend or file storage and returns a shared model. Finally, the state is updated and the interface shows the result. Screens do not store secrets or call backend endpoints directly.

### 2.4.8 Main Cross-Platform UI Flow

The main interaction between the mobile and web interfaces is summarized as follows:

1. The photographer searches for a Film Lab and creates an order in the mobile application.
2. Film Lab staff review the order and update its processing stages in the web portal.
3. Status notifications are displayed in the Photographer Application.
4. The Film Lab uploads and publishes scan files after quality checking.
5. The photographer previews, downloads, and organizes the scans in the Digital Film Archive.
6. Reports or disputes are transferred to the Administration Portal for review.

### 2.4.9 Frontend Sequence Summaries

Four target frontend sequence diagrams are documented in docs/MobileWeb.md. The following summaries describe the proposed shared-API interactions:

1. **Create service order:** the Photographer App validates the booking, sends it to the Platform API, and displays the returned order code.
2. **Update processing status:** the Film Lab Portal sends the next status to the API, which records the history and notifies the photographer.
3. **Publish scans:** the Film Lab Portal uploads files to storage, saves metadata through the API, and makes the files available in the mobile application.
4. **Moderate a listing:** the Administration Portal retrieves the report and evidence, records the decision, updates the Marketplace, and writes an audit event.

### 2.4.10 Prototype Screens and Presentation Plan

The implemented prototype screens provide concrete interface examples: Photographer Home, Lab details and booking preview, Orders, Archive, Marketplace, the Film Lab dashboard, processing board, scan review, and administration queues. Source paths and acceptance steps are listed in docs/AcceptanceTraceability.md. These interactive prototypes are distinct from a separate low-fidelity wireframe package; no additional wireframe files are claimed.

During the presentation, the UI flow should be introduced first, followed by one mobile example, one web portal example, the component architecture, and the scan delivery sequence. The recommended speaking time for this section is four to five minutes.

## Chapter 3

# Prototype Handover and Verification

### 3.1 Repository Deliverables

Table 3.1: Evidence and implementation boundaries

| Deliverable       | Repository evidence                                       | Boundary                                                                                                            |
|-------------------|-----------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Mobile app        | apps/mobile/lib/,<br>apps/mobile/test/                    | Flutter source and demonstration data; device build and shared API integration require separate evidence.           |
| Web portals       | apps/web/src/,<br>package.json                            | apps/web/React/TypeScript interface for lab and admin workflows; local demonstration state.                         |
| Business API      | backend/server.js, backend/<br>authMiddleware.js          | Express endpoints with PostgreSQL dependency; configured database and integration tests are required.               |
| AI prototype      | ai_service/,<br>openapi.json                              | ai_service/Local recommendation, retrieval, grounded extraction, and scan-quality heuristics.                       |
| AI validation     | tests/, scripts/evaluate_ai.<br>py, scripts/smoke_test.py | Automated tests and reproducible evaluation scripts; synthetic/local results do not demonstrate production quality. |
| Deployment design | Dockerfile, infra/azure/,<br>.github/workflows/           | Container, infrastructure, and workflow definitions; presence of files is not proof of a live deployment.           |
| Design report     | CNPM_Report.tex, sections/,<br>docs/                      | Consolidated requirements, responsibilities, architecture, interface flows, and submission evidence.                |

### 3.2 Demonstration Procedure

1. Install the web dependencies in apps/web/ with `npm ci`; run `npm run build` and `npm run dev`. Open the local address printed by Vite.

2. Demonstrate Film Lab and Admin portal navigation, order search/filtering, processing stages, marketplace/archive views, and available form validation. Describe each operation as a local prototype action unless a verified API response and persistent database result are shown.
3. For mobile, install a compatible Flutter SDK, run `flutter pub get`, `flutter analyze`, and `flutter test` inside `apps/mobile/`. Launch a supported device target with `flutter run`. Source inspection alone is not a device test.
4. From the repository root, install Python project dependencies and run `python -m unittest discover -s tests -v`. Start the AI adapter with `python -m ai_service.api`; use `/health`, `/ready`, and `/openapi.json` to inspect its contract.
5. Present one supported photography question, one unsupported question, and one recommendation request. Explain the local knowledge/data boundary and show how insufficient evidence is reported.
6. Open this report and the acceptance/submission documents in `docs/`. Relate each demonstrated feature to the relevant requirement and work item.

### 3.3 Acceptance and Remaining Integration Work

For each requirement, record its identifier, owner, code or document path, test procedure, observed result, and unresolved dependency. A feature is complete only when its agreed acceptance criteria have supporting evidence. Interface mockups, implemented local interactions, automated unit tests, and verified end-to-end operations are distinct evidence levels.

Before claiming the complete platform is operational, the team must verify:

- database schema and repeatable migrations, seeded non-sensitive test data, configured secrets, and authorization for each protected resource;
- mobile/web authentication and shared API integration, including cross-user isolation, failure states, and persistence after reload;
- private cloud uploads, ownership checks, scan publication and downloads;
- real payment, logistics, and notification adapters, or an explicitly documented decision to exclude them from the assessed prototype;
- supported-device mobile builds, end-to-end tests, accessibility checks, and performance/recovery measurements against the proposed targets;
- deployment health, backup and restore verification, and reproducible release configuration before calling the system production ready.

### 3.4 Contribution and Submission Evidence

The source repository is the primary record of implemented changes. Each meaningful commit should describe a coherent change and, where available, reference its Jira issue. Jira items should link to the resulting code, document, test evidence, or demonstration and describe unresolved dependencies. Only move an item to Done when its actual acceptance criteria are satisfied.

Contribution evidence should connect each member's requirements, implemented artifacts, tests and reviews. Commit counts alone do not measure completeness or quality; planned work must remain distinct from completed work.

### 3.5 Verification Snapshot

The submission snapshot was checked on Windows using Node.js 24.19 and the bundled Python environment on 17 September 2026. The web production build and TypeScript compilation succeeded. The automated suites passed 41 AI tests, 9 web-domain tests, and 11 backend authorization/validation tests. Backend tests use an isolated database stub; no live PostgreSQL or cloud deployment is claimed.

| Area         | Verification evidence                                                                                                           | Result / limitation                                         |
|--------------|---------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| AI           | Recommendation constraints, retrieval, refusal on insufficient evidence, scan quality and HTTP contracts                        | 41 tests passed; local and synthetic datasets.              |
| Web domain   | Order transitions, service validation, scan boundaries, publication, approval/moderation, CSV safety and saved-state round trip | 9 tests passed.                                             |
| Business API | Shared JWT key, malformed/expired/forged tokens, role checks, private order listing, safe login and server-derived order prices | 11 tests passed; database stub.                             |
| React build  | TypeScript and Vite production output                                                                                           | Passed.                                                     |
| Mobile       | Source and navigation/widget test inspected; booking estimate and demo wording corrected                                        | Flutter SDK unavailable locally; device build not verified. |
| Report       | LaTeX sources consolidated and compiled using Tectonic                                                                          | PDF generated and rendered pages visually reviewed.         |

### 3.6 Requirement-to-Code Examples

#### US-03 / SCRUM-29: inspect an order

A lab operator can search by order, customer or service and filter by status. `apps/web/src/App.tsx` contains the order table, selection and detail view. Keyboard Enter selects a row.

#### US-04: advance a workflow

`apps/web/src/domain.ts` permits only the next stage. Completed orders cannot advance, and scan orders at quality check require reviewed publication before completion.

#### US-05: deliver scans

JPEG/TIFF extension and MIME checks, positive size, 50 MB per-file limit, duplicate filenames and a 100-file limit are enforced. The demo saves private metadata after quality and release confirmation; actual file bytes remain on the device.

#### US-08 and US-09 / SCRUM-30: administration

Incomplete lab applications cannot be approved. Rejections and moderation decisions require reasons. Repeated final decisions are rejected and local audit entries record changes. Browser audit data is not tamper-proof server evidence.

#### Security boundary

The Express API shares one configured signing and verification secret, restricts user/schema inspection to administrators, and filters non-admin orders by owner. Order totals come from the service catalogue.

### 3.7 Workflow and Sequence Diagrams

The diagrams below distinguish implemented local interactions from the target integration design. The complete story/rule matrix and manual checklist are in docs/AcceptanceTraceability.md and docs/SubmissionChecklist.md.

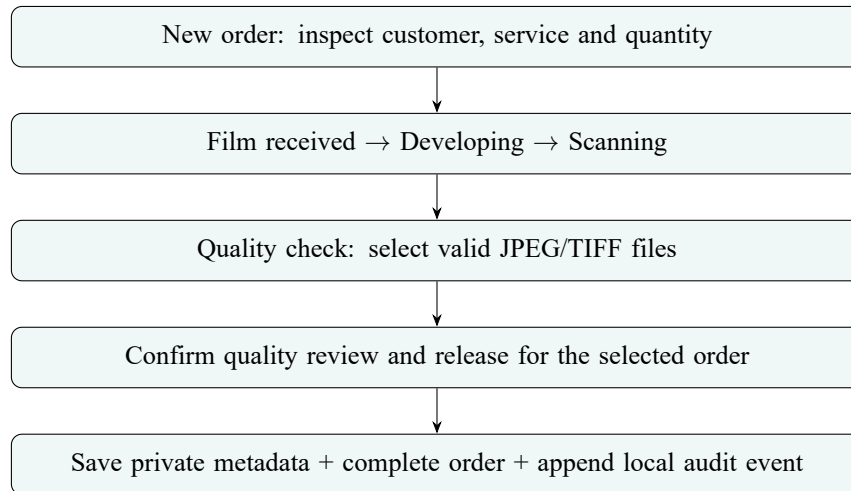


Figure 3.1: Implemented local web workflow; no cloud upload or customer message is sent

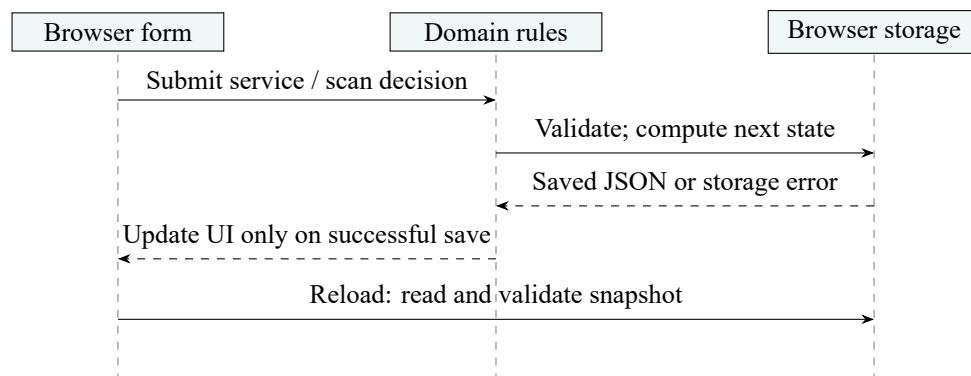


Figure 3.2: Implemented state persistence, validation and error handling in the web demo

### 3.8 Known Limits and Demonstration Talking Points

The assessed implementation is a documented prototype with executable local interactions and testable backend/AI modules. It is not a completed production marketplace. Shared frontend authentication, live database migration verification, cross-device synchronization, cloud scan storage, payment/logistics adapters, mobile device testing and deployment remain explicit integration tasks.

For a five-minute demonstration: introduce the problem and actors; show order search and a valid status transition; demonstrate a service edit that survives reload; show scan validation and private publication metadata; demonstrate an admin decision and its audit event; finish with the test summary and the requirement-to-code matrix. Explain that local role switching is a demonstration control and that backend authorization must independently enforce permissions.

### 3.9 Measurable Quality Acceptance Plan

The following thresholds are proposed engineering targets for team review, not an official lecturer rubric, measured service levels or completed tests. They make the broad non-functional requirements assessable in a later integration milestone.

| ID / area              | Proposed acceptance criterion                                                                                                  | Method / current evidence                                                                                |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| NFR-01 Security        | All protected endpoints reject absent/invalid tokens; forbidden role/resource access cannot read or mutate data.               | Negative HTTP cases and cross-user tests. Eleven backend tests provide partial evidence, with a DB stub. |
| NFR-02 Performance     | With 10,000 seeded labs and 50 concurrent clients, discovery API p95 below 2 s and error rate below 1% during a 10-minute run. | Record hardware, dataset, warm-up, latency distribution and errors. Not run.                             |
| NFR-03 Reliability     | Restart after an acknowledged order write preserves that order; retrying the same booking key produces one order.              | Kill/restart plus duplicate-request integration test. Persistence/idempotency not verified.              |
| NFR-04 Recovery        | Proposed development recovery objective: at most 24 h data loss and restoration within 4 h.                                    | Restore a backup into an isolated DB and verify row/file ownership counts. Not run.                      |
| NFR-05 Privacy         | Customer A cannot obtain customer B's scan metadata or file bytes; new deliveries default private.                             | API and storage authorization tests, including expired links. Only local metadata privacy is tested.     |
| NFR-06 Usability       | Core forms usable by keyboard; required actions reachable at 360 px and 1280 px widths without clipped controls.               | Manual focus, labels, validation, narrow-screen review. No complete accessibility audit claimed.         |
| NFR-07 Integrity       | Concurrent reservations cannot oversell; invalid quantity, lab/service mismatch and dates are rejected.                        | API unit tests cover part of validation; concurrent DB transactions remain untested.                     |
| NFR-08 Maintainability | A fresh checkout installs from lockfiles and passes TypeScript build and deterministic unit suites.                            | npm ci and web build passed; 61 automated tests passed. Flutter execution is pending.                    |
| NFR-09 AI behavior     | Unsupported questions disclose insufficient evidence; recommendations respect explicit hard constraints.                       | Existing local AI tests cover constrained behavior; results are not a general model-accuracy benchmark.  |

### 3.10 Acceptance Traceability

Detailed stories and manual TC identifiers are maintained in docs/AcceptanceTraceability.md and docs/SubmissionChecklist.md. The matrix below is a compact assessment guide; “local” always means one browser using demonstration data.

| Requirement      | Artifact / Jira                            | Evidence and status                                                                               |
|------------------|--------------------------------------------|---------------------------------------------------------------------------------------------------|
| UC-01 / US-01    | Mobile Home; SCRUM-5/28                    | Search/filter source exists. Flutter runtime unverified; AI tests are independent of mobile.      |
| UC-02 / US-02    | Mobile preview, POST /orders; SCRUM-6/28   | Preview pricing corrected; API quantity/price tested with stub. Full booking flow not integrated. |
| UC-03 / US-03/04 | Web Orders and domain; SCRUM-8/29          | Search/details and permitted local transitions; domain tests pass. No mobile synchronization.     |
| UC-04 / US-05/06 | ScanDelivery, Archive; SCRUM-9/29/31       | Validation/publication logic tested; picker automation blocked. No cloud delivery/download proof. |
| UC-05 / US-07/08 | Marketplace, Moderation; SCRUM-10/30/31    | Local listing decisions and audit rules tested. No completed purchase/payment.                    |
| UC-06 / BR-08    | FunctionalRequirements; course API         | Six-flow specification exists. Registration, capacity and article discussion remain planned.      |
| US-09 / BR-07    | ApprovalsPage; SCRUM-30                    | Incomplete applications blocked; reason requirements tested. No server approval workflow.         |
| US-10            | Theme/styles and form components; SCRUM-27 | Explicit empty/error/focus states; complete usability assessment pending.                         |
| US-11 / BR-09    | Mobile assistant and Python AI; SCRUM-28   | Mobile responses are samples. Python tests cover independent retrieval/recommendation behavior.   |
| US-12            | Report, diagrams, matrix; SCRUM-4/26/32    | Source paths, use cases, schema and verification connected in the document package.               |

### 3.11 Release Checklist and Risks

1. Review this PDF with the source requirements and any official marking rubric.
2. Check that the main branch contains the final source and documentation; record the exact release commit rather than relying on a local folder name.
3. Install clean dependencies and run the documented component commands; preserve failures and unverified integrations in the handover notes.
4. Rehearse the six-minute local demo and explain the role/storage boundaries.
5. Link GitHub evidence on the relevant Jira issues; only close an issue when its own acceptance criteria have been met.
6. Confirm the submission destination and submit through the course's required channel. A GitHub merge does not establish submission to the lecturer.

The most significant remaining risks are frontend/API contract mismatch, unverified mobile build and database setup, and absent cloud delivery and transaction services. Mitigation is to present the tested

prototype scope accurately, prioritize a single authenticated booking-to-delivery integration, then add resource authorization and failure-recovery tests before deployment. The team should obtain the official rubric to determine whether a proposal, prototype or fully integrated system is required for this milestone.

## 3.12 Glossary and Evidence Sources

### Film Lab

A provider that receives physical film, processes it and optionally scans/prints the results.

### Scan delivery

Release of digital output linked to an order; selecting a local file alone is not delivery.

### Digital Film Archive

A customer's organized collection of scan files and metadata, with ownership and privacy controls.

### RBAC / ownership

Role permissions determine the class of allowed action; resource ownership limits which customer's or lab's records it can affect.

### Prototype

A limited implementation for evaluating interactions and design, with explicitly documented data and integration boundaries.

### Traceability

A link from a requirement to its design, source implementation, test or review result and work item.

Primary evidence for this report is the project repository and its executable tests, rather than external benchmark claims:

- Repository: <https://github.com/thanhtoan101/CNPM>.
- Requirement integration: <https://github.com/thanhtoan101/CNPM/pull/9>.
- Mobile/Web integration: <https://github.com/thanhtoan101/CNPM/pull/8>.
- Completion review: <https://ut-team-fy1tb246.atlassian.net/browse/SCRUM-39>.
- Source specifications: docs/FunctionalRequirements.md, docs/MobileWeb.md, backend/schema.sql, backend/README.md.
- Reproduction and test boundaries: README.md, docs/Verification.md, apps/web/test/domain.test.ts, backend/test/auth.test.js, tests/.